딥러닝 파이토치 2장: 실습 환경 설정과 파이토치 기초

Status	In progress
Assignee	시현 이
Due date	@03/10/2025
Task type	개념및코드필사
Description	딥러닝 파이토치 교과서 1-7장 개념 및 코드 필사

2.1 파이토치 개요

파이토치: 딥러닝 프레임워크. 루아(Lua) 언어로 개발됐던 토치를 페이스북에서 파이썬 버전으로 내놓은 것.

파이썬 기반의 과학 연산 패키지로

- 넘파이를 대체하면서 GPU를 이용한 연산이 필요한 경우
- 최대한 유연성과 속도를 제공하는 딥러닝 연구 플랫폼이 필요한 경우

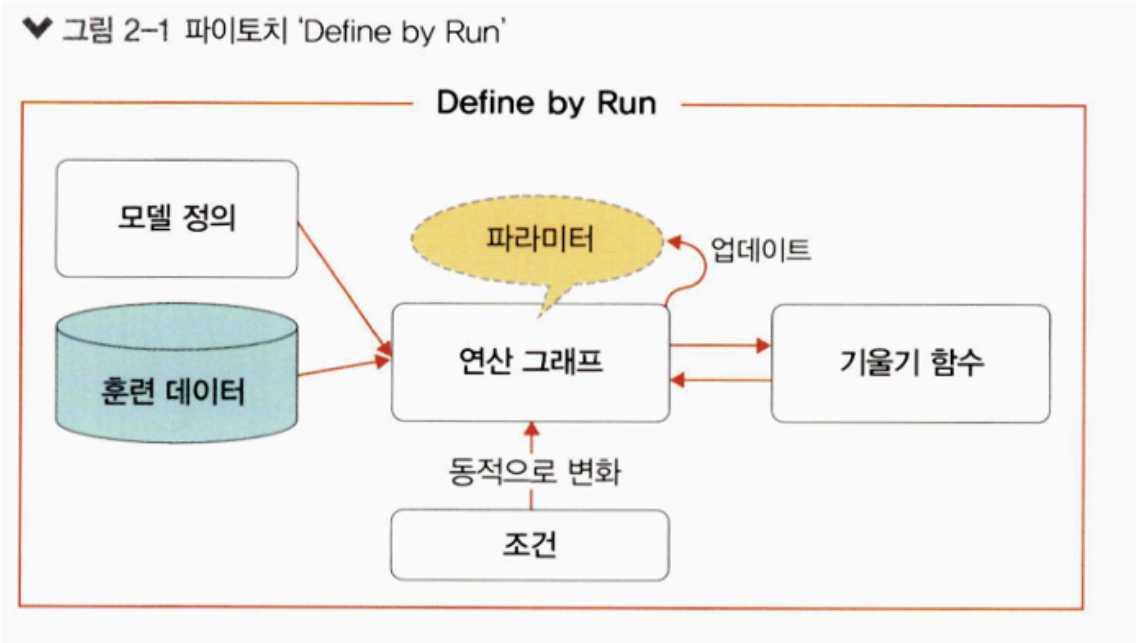
를 대상으로 함

2.1.1 파이토치 특징 및 장점

: GPU에서 텐서 조작 및 동적 신경망 구축이 가능한 프레임워크

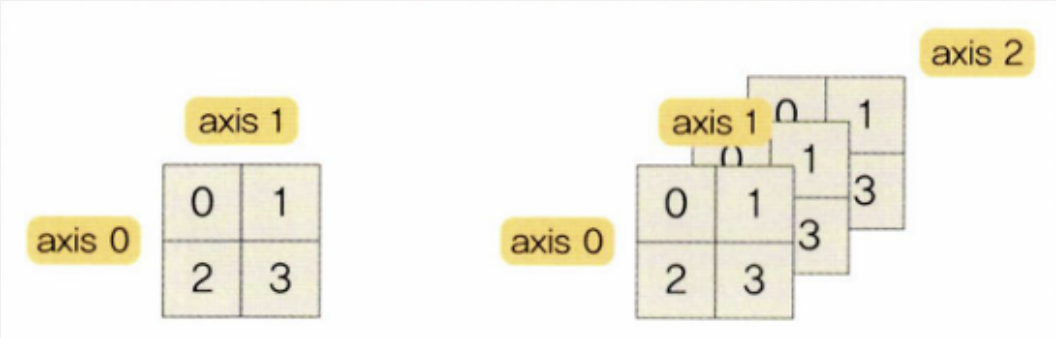
- GPU(Grapics Processing Unit)
  - 연산 속도를 빠르게 하는 역할
  - 기울기 미분 계산 시, 빠른 계산 가능
  - CUDA,cuDNN 이라는 API 통해 GPU를 연산에 사용할 수 있음
  - 병렬 연산에서 GPU 속도는 CPU의 속도보다 훨씬 빠름

- 텐서(Tensor)
  - 파이토치의 데이터 형태
  - 단일 데이터 형식으로 된 자료들의 다차원 행렬
  - 간단한 명령어(변수.cuda() 추가)를 사용해서 GPU로 연산이 가능
- 동적 신경망
  - 훈련을 반복할 때마다 네트워크 변경이 가능한 신경망
    - 학습 중에 은닉층을 추가하거나 제거하는 등 모델의 네트워크 조작이 가능
  - 연산 그래프를 정의하는 것과 동시에 값도 초기화되는 'Define by Run' 방식을 사용
    - 연산 그래프와 연산을 분리해서 생각할 필요X



벡터, 행렬, 텐서
인공지능에서의 데이터 == 벡터

- 1차원 축(행) == axis 0 == 벡터
- 2차원 축(열) == axis 1 == 행렬
- 3차원 축(채널) == axis2 == 텐서



행렬: 복수의 차원을 가지는 데이터 레코드의 집합

- 하나의 데이터 레코드를 벡터 단독으로 나타냄

$$x_1 = \begin{bmatrix} 1.1 \\ 2.7 \\ 3.3 \\ 0.2 \end{bmatrix} \quad x_2 = \begin{bmatrix} 4.5 \\ 1.2 \\ 0.7 \\ 3.5 \end{bmatrix}$$

- 복수의 데이터 레코드 집합을 행렬로 나타낼 때는 하나의 데이터 레코드가 하나의 행으로 표기됨

$$X = \begin{bmatrix} 1.1 & 2.7 & 3.3 & 0.2 \\ 4.5 & 1.2 & 0.7 & 3.5 \end{bmatrix}$$

$$X = \begin{bmatrix} x_{11} & x_{12} & x_{13} \\ x_{21} & x_{22} & x_{23} \\ x_{31} & x_{32} & x_{33} \end{bmatrix}$$

⇒ 행의 일반적인 표기

- **텐서**: 행렬의 다차원 표현. 같은 크기의 행렬이 여러개 묶여 있는 것으로 표현

$$X = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1n} \\ x_{21} & x_{22} & \cdots & x_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ x_{m1} & x_{m2} & \cdots & x_{mn} \end{bmatrix}$$

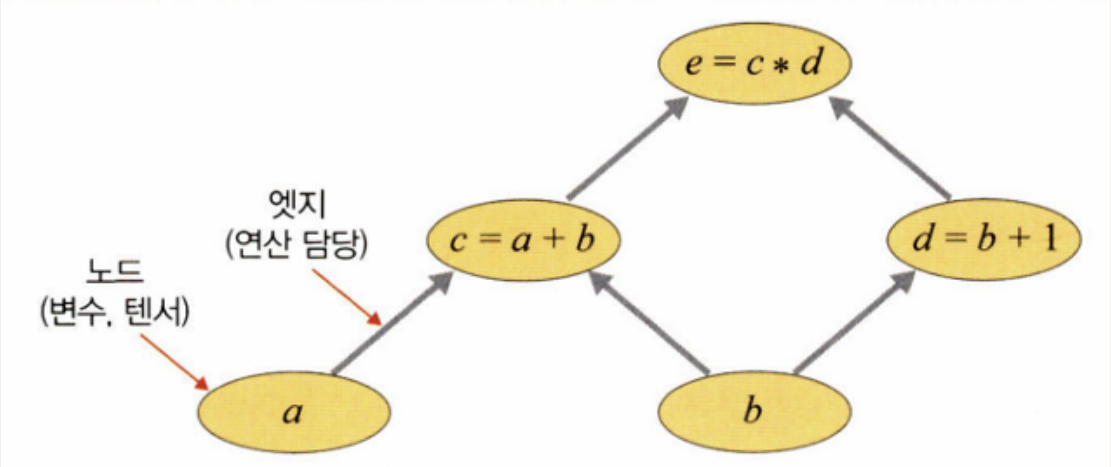
- 파이토치에서 텐서 표현하기 위해서 `torch.tensor()`를 사용

```
#tensor 표현
import torch
torch.tensor([[1.,-1.], [1.,-1.]])

#생성된  텐서의  형태
tensor([[1.,-1.],
                [1.,-1.]])
```

연산 그래프
: 방향성이 있으며 변수(ex.tensor)를 의미하는 노드와 연산을 담당하는 엣지로 구성됨

- 노드는 변수(a,b)를 가지고 있으며 각 계산을 통해 새로운 텐서(c,d,e)를 구성



신견망은 연산 그래프를 이용해 계산 수행
⇒ 네트워크가 학습될 때 손실 함수의 기울기가 가중치와 바이어스를 기반으로 계산되며, 이후 경사 하강법을 사용해 가중치가 없데이트됨. 이대, 연산 그래프를 이용해 이 과정이 효과적으로 수행됨

파이토치의 장점

- 단순함(효율적 계산)
  - 파이썬 환경과 쉽게 통합
  - 디버깅이 직관적, 간결
- 성능(낮은 CPU 활용)
  - 모델 훈련을 위한 CPU 사용률이 텐서플로와 비교해 낮음
  - 학습 및 추론 속도가 빠르고 다루기 쉬움
- 직관적인 인터페이스
  - 텐서플로처럼 잦은 API 변경이 없어 배우기 쉬움

2.1.2 파이토치의 아키텍처

3개의 계층으로 설명 가능

파이토치 API

: 사용자가 이해하기 쉬운 API 제공해 텐서에 대한 처리와 신경망을 구축하고 훈련할 수 있도록 도움

: 사용자 인터페이스를 제공하지만 실제 계산은 수행X

→ 대신, C++로 작성된 파이토치 엔진으로 그 작업을 전달하는 역할

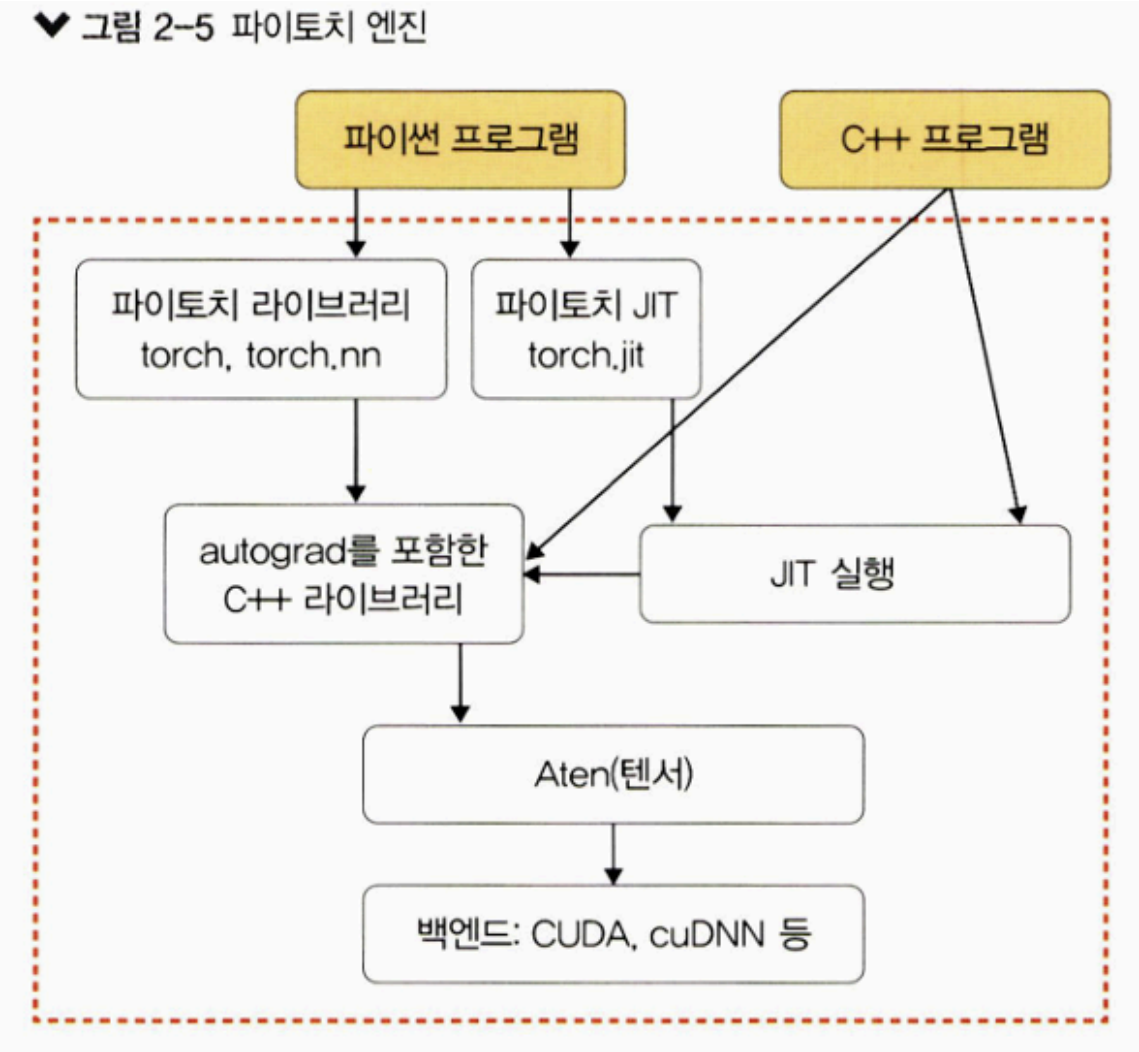
[파이토치 패키지]

- torch: GPU를 지원하는 텐서 패키지
  - 다차원 텐서 기반으로 다양한 수학적 연산 가능
- torch.autograd: 자동 미분 패키지
  - 미분 계산을 효율적으로 처리함으로써 신경망 변경 시 신경망 구축을 처음부터 할 필요가 없어짐
  - 연산 그래프가 즉시 계산되기 때문에 다양한 신경망 적용 가능
- torch.nn: 신경망 구축 및 훈련 패키지
  - torch.nn을 사용할 경우 신경망을 쉽게 구축하고 사용할 수 있음
  - 합성곱 신경망, 순환 신경망, 정규화 등이 포함
- torch.multiprocessing: 파이썬 멀티프로세싱 패키지
  - 프로세스 전반에 걸쳐 메모리 공유 가능
  - 서로 다른 프로세스에서 동일한 데이터에 대한 접근 및 사용 가능
- torch.utils: DataLoader 및 기타 유틸리티를 제공하는 패키지
  - 모델에 데이터를 제공하기 위한 torch.utils.data.DataLoader 모듈을 주로 사용
  - 병목 현상을 디버깅하기 위한 torch.utils.bottleneck
  - 모델 또는 모델의 일부를 검사하기 위한 torch.utils.checkpoint 등 의 모듈

파이토치 엔진

:다차원 텐서 및 자동 미분을 처리

- Autograd C++, Aten C++, JIT C++, Python API로 구성돼 있음
- Autograd C++ : 가중치, 바이어스를 업데이트하는 과정에서 필요한 미분을 자동으로 계산해 주는 역할
- Aten C++: C++ 텐서 라이브러리를 제공
- JIT C++ :계산을 최적화하기 위한 JIT(just-in-time) 컴파일러
- 파이토치 엔진 라이브러리는 c++로 감싼 다음 Python API 형태로 제공됨



연산 처리

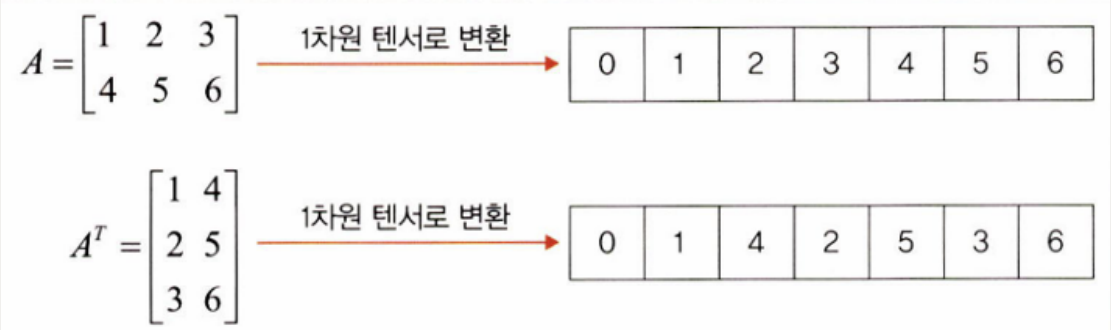
: C 또는 CUDA 패키지는 상위의 API에서 할당된 거의 모든 계산을 수행

- CPU와 GPU(TH(토치), THC(토치 CUDA))를 이용해 효율적인 데이터 구조, 다차원 텐서에 대한 연산 처리

텐서를 메모리에 저장
텐서는 그것이 1차원이든 N차원이든 메모리에 저장할 때는 1차원 배열 형태가 됨.
→ 1차원 형태여야만 메모리에 저장할 수 있음
→ 변환된 1차원 배열을 '스토리지(storage)'라고 함

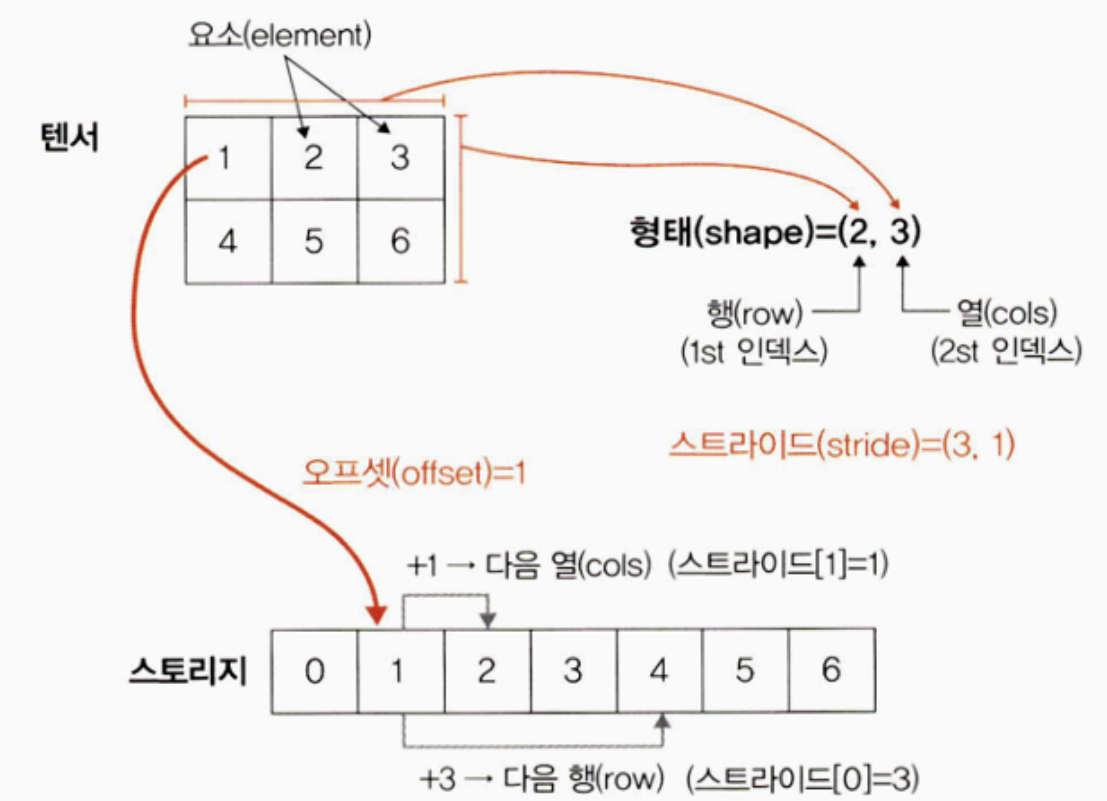
- **오프셋(offest)**: 텐서에서 첫번째 요소가 스토리지에 저장된 인덱스
- **스트라이드(stride)**: 각 차원에 따라 다음 요소를 얻기 위해 건너뛰기가 필요한 스토리지의 요소 개수. 즉, 스트라이드는 메모리에서의 텐서 레이아웃을 표현하는 것으로 이해. 요소가 연속적으로 저장되기 때문에 행 중심으로 스트라이드는 항상 1

<전치행렬과 오프셋, 스트라이드의 관계성>
행,열의 개수가 다른 텐서 A와 A^T 생성한 후, 이들을 1차원 배열로 변환



서로 다른 형태를 갖지만 스토리지의 값들은 서로 같음
⇒ 이 둘을 구분하기 위해 오프셋과 스트라이드 개념을 사용

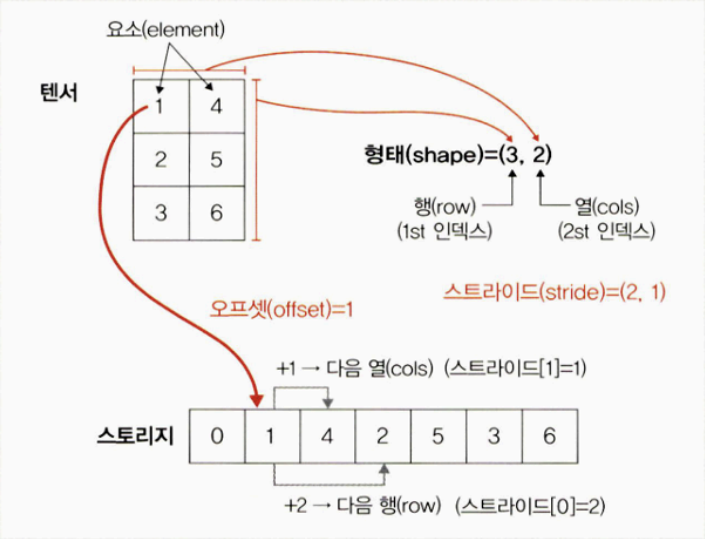
A를 1치원을 변환했을 때, 스토리지에서 2를 얻기 위해선 1에서 1칸을 뛰어넘어야하고, 4를 얻기 위해서는 3을 뛰어넘어야함
⇒ 텐서에 대한 스토리지의 스트라이드는 **(3,1)**



A^T를 1차원으로 변환했을 때, 스토리지에서 4를 얻기 위해서는 1에서 1칸을 뛰어넘어야하고, 2를 얻기 위해서는 2를 뛰어넘어야함
⇒ 스토리지의 스트라이드는 **(2,1)**

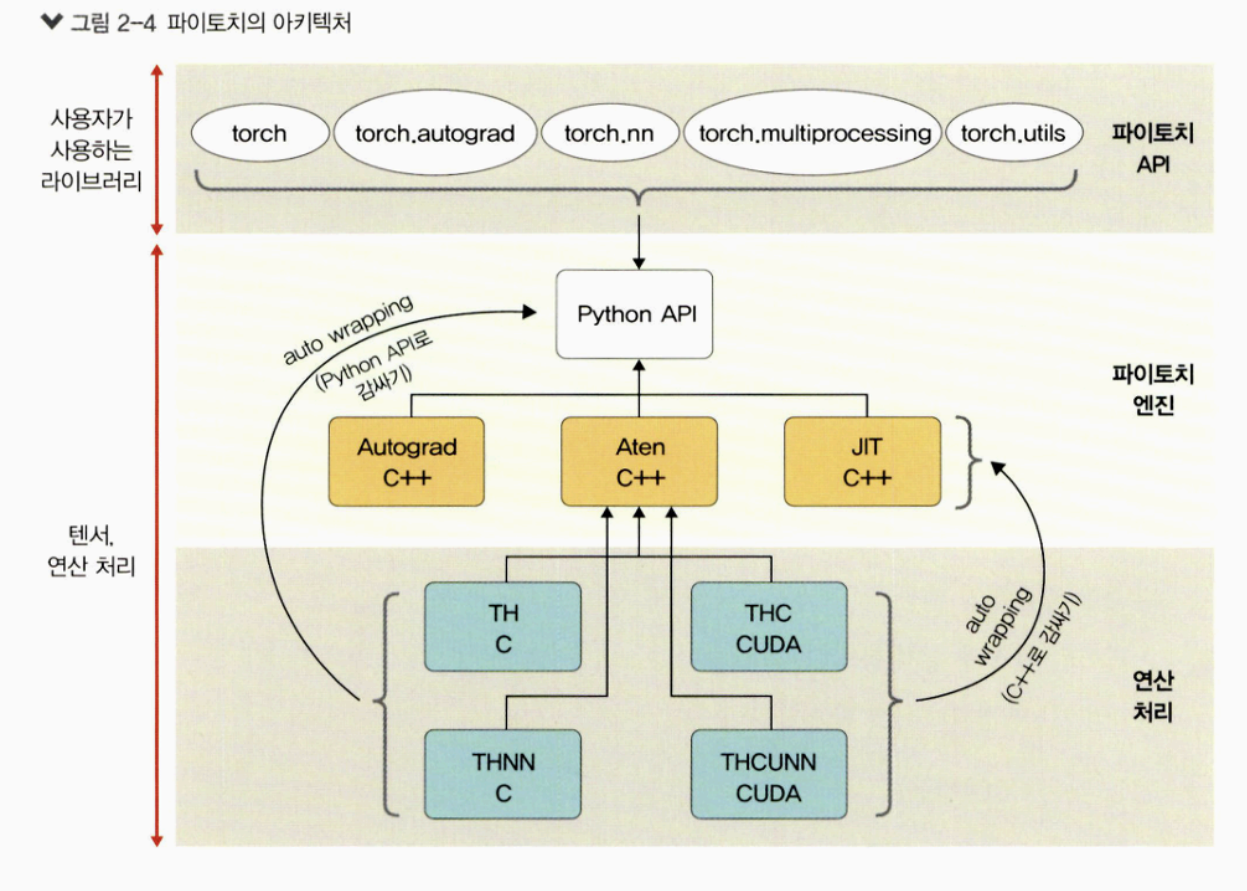
반면에 A의 전치 행렬은 좀 다릅니다. 다음 그림의 스토리지에서 4를 얻기 위해서는 1에서 1칸을 뛰어넘어야 하고, 2을 얻기 위해서는 2를 뛰어넘어야 합니다. 따라서 텐서에 대한 스토리지의 스트라이드는 (2, 1)입니다.

❤ 그림 2-8 A의 전치 행렬을 1차원으로 변환



이와 같이 오프셋과 스트라이드는 데이터 자체가 아닌 행렬/텐서를 구분하기 위해 사용합니다.

이와 같이 오프셋과 스트라이드는 **데이터 자체가 아닌 행렬./텐서를 구분하기 위해 사용**



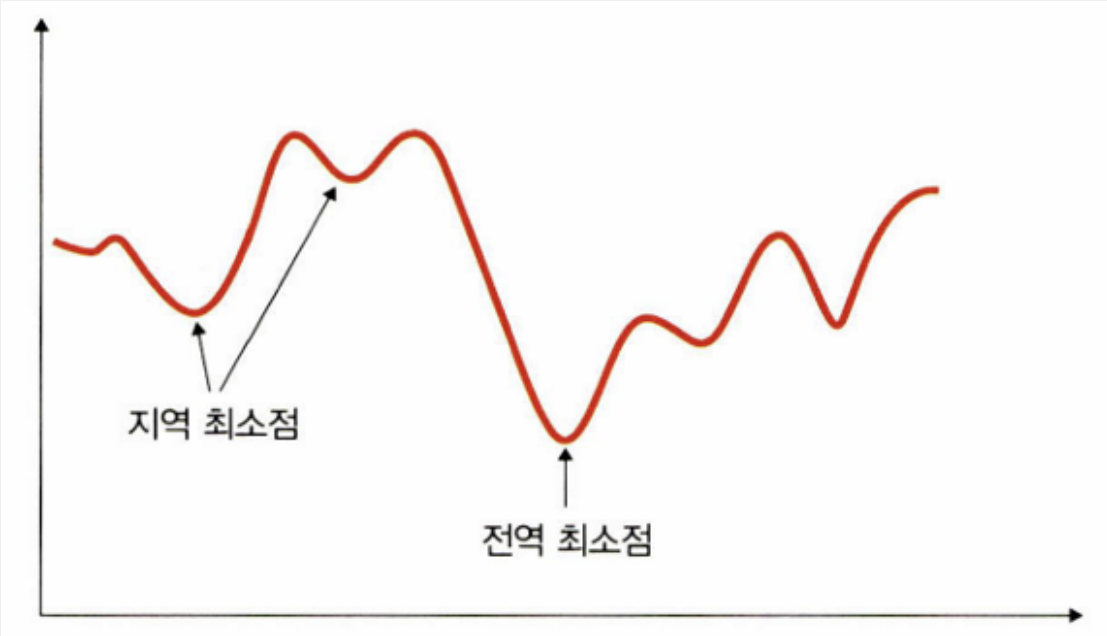
2.2 파이토치 기초 문법

2.2.4 모델의 파라미터 정의

- 손실함수
  - 학습하는 동안 출력과 실제 값 같의 오차를 측정
  - **BCELoss**: 이진 분류를 위해 사용
  - **CrossEntropyLoss**: 다중 클래스 분류를 위해 사용
  - **MSELoss**: 회귀 모델에서 사용
- 옵티마이저
  - 데이터와 손실 함수를 바탕으로 모델의 업데이트 방법을 결정
  - 주요 특성
    - optimizer는 step() 메서드를 통해 전달받은 파라미터를 업데이트

        - 모델의 파라미터별로 다른 기준을 적용
        - torch.optim.Optimizer(params, defaults)는 모든 옵티마이저의 기본이 되는 클래스
        - zero_grad() 메서드는 옵티마이저에 사용된 파라미터들의 기울기를 0을로 만듦
        - torch.optim.lr_scheduler 는 에포크에 따라 학습률 조절
        - 옵티마이저에 사용되는 종류
            - optim.Adadelta, optim.Adagrad, optim.Adam, optim.SparseAdam, optim.Adamax
            - optim.ASGD, optim.LBFGS
            - optim.RMSProp, optim.Rprop, optim.SGD
- 학습률 스케줄러
    - 미리 지정한 횟수의 에포크를 지날 때마다 학습률을 감소시켜줌.
    - 학습 초기에는 빠른 학습 → 전역 최소점 근처에 다다르면 학습률을 줄여서 최적점을 찾아갈 수 있도록 함
    - 종류
        - optim.lr_scheduler.LambdaLR: Lambda 함수를 이용해 그 함수의 결과를 학습률로 설정
        - optim.lr_scheduler.StepLR: 특정 단계마다 학습률을 감마 비율만큼 감소
        - optim.lr_scheduler.ExponentialLR: 에포크마다 이전 학습률에 감마만큼 곱함
        - optim.lr_scheduler.CosineAnnealingLR: 학습률을 코사인 함수의 형태처럼 변화시킴.
        - optim.lr_scheduler.ReduceLR0nPlateau: 학습이 잘되는지 여부에 따라 동적으로 학습률 변화시킴
- 지표(metrics)
    - 훈련과 테스트 단계를 모니터링

전역 최소점과 최적점
전역 최소점(global minimum): 오차가 가장 작을 때의 값을 의미. 최적점
지역 최소점(local minimum): 전역 최소점을 찾아가는 과정에서 만나는 홀.

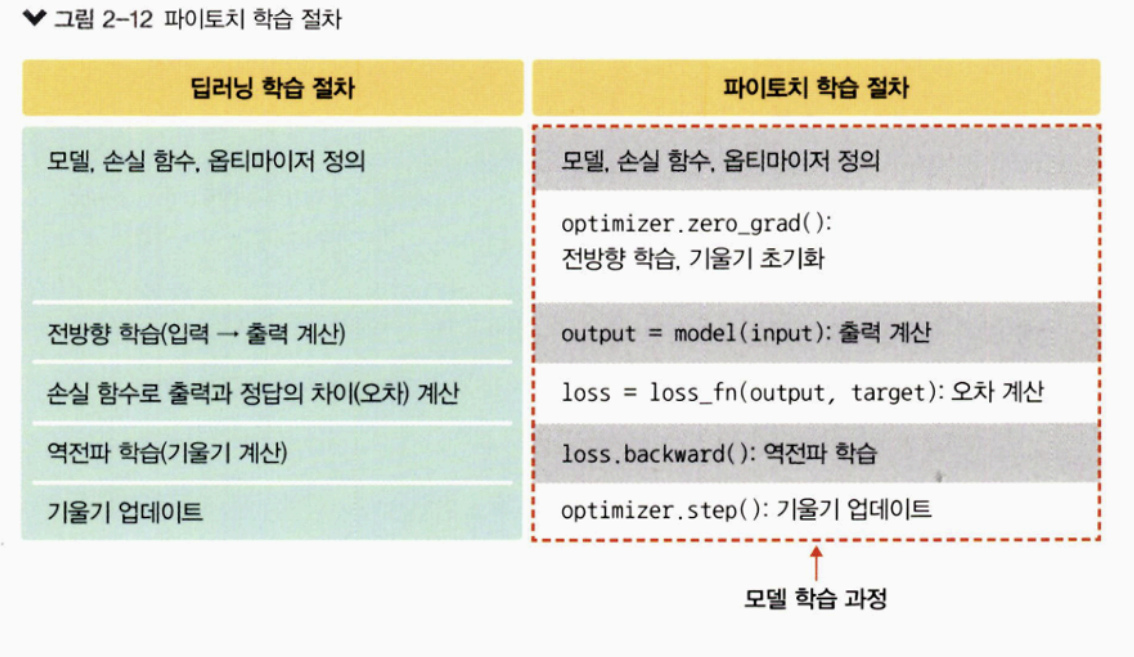


2.2.5 모델 훈련

모델 학습 **==** y = wx+b라는 함수에서 w와 b의 적절한 값을 찾는다는 의미(전역 최소점)

<훈련 방법>

1. optimizer.zero_grad() 메서드를 이용해 기울기를 초기화
    a. loss.backward() 메서드 사용해 기울기 값 계산 ⇒ 새로운 기울기 값이 이전 기울기 값에 누적해 계산됨
    b. 순환 신경망(RNN) 모델을 구현할 때 효과적이지만 누적 계산이 필요하지 않는 모델에 대해서는 불필요
        i. 기울기 값에 대해 누적 계산이 필요하지 않을때는 입력 값을 모델에 적용하기 전에 optimizer.zero_grad() 메서드를 호출해 미분 값이 누적되지 않게 초기화



2. loss.backward() 메서드를 이용해 기울기를 자동 계산
   a. loss.backward()는 배치가 반복될 때마다 오차가 중첩적으로 쌓이게 되므로 매번 zero_grad()를 사용해 미분 값을 0으로 초기화
   b. 예시 코드

```
for epoch in range(100):
    yhat = model(x_train)
    loss= criterion(yhat, y_train)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

2.3 실습 환경 설정

colab 사용 예정.

2.4 파이토치 코드 맛보기

딥러닝 분류 모델의 성능 평가 지표

- True Positive: 모델과 실제값 '1'로 일치
- True Negative: 모델과 실제값 '0'으로 일치
- False Positive: 모델은 '1', 실제값은 '0'인 경우. Type1 오류
- False Negative: 모델은 '0', 실제값은 '1', Type2 오류

정확도

전체 예측 건수에서 장답을 맞힌 건수의 비율

$$\frac{True\ Positive + True\ Negative}{True\ Positive + True\ Negative + False\ Positive + False\ Negative}$$

재현율

실제 정답이 1일 때, 모델도 1로 예측한 비율

$$\frac{True\ Positive}{True\ Positive + False\ Negative}$$

정밀도

모델이 1이라고 예측한 것 중에서 실제로 정답이 1인 비율

$$\frac{True\ Positive}{True\ Positive + False\ Positive}$$

F1-score

정밀도와 재현율의 조화 평균을 이용한 지표

$$2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Supporting files

https://colab.research.google.com/drive/1YRACOtbVJ2VsPWf69cEhucm4WLadlBCq?usp=sharing